

# Handoff #7: three decision points, one answer

Every experiment is now run: agent control (handoffs #4-#6), and since #6 the cache guard and the router, built and run end to end. Part I explains the whole project from scratch; Part II is the usual handoff.

## ***TL;DR***

- The question: can a cheap model that only decides (Jev, about \$0.00002 per decision) save money against a frontier model (GPT-5.6) that generates everything?
- The answer, across three decision points: a standalone decider pays off when its decision replaces generation and it can see something the cheap alternative can't. Not when generation happens anyway, and not when the knowledge it needs isn't in its input.
- Agent control: no. GPT-5.6 deciding inside its own single call is cheapest (reward 0.79 vs Jev+G 0.74).
- Cache guard: yes, where it matters. On GSM-Plus Jev is right 93.3% vs 66% for every similarity method, and cuts cost 28% where they cut nothing. A GPT-5.6 guard costs more than not caching (136%).
- Router: barely. A router trained on the small model's own record recovers 65% of the quality gap vs Jev's 61% (random mix 50%); GPT-5.6 is at chance (53%).
- Open: your 100-case label audit (cache guard), and whether to run one more agent-control experiment.

## ***Your first 5 minutes***

```
git checkout main && git pull # everything is on main
less results/replay/cacheguard-paired.md # cache-guard claims, paired CIs
less results/replay/router-routerbench/report/summary.md # router table + CIs
less DECISIONS.md # rows dated 2026-10-05
```

# Part 1 · The project, explained

## What we are testing

LLM systems make many small decisions before (or instead of) generating text: which tool to call, whether a cached answer can be reused, which model should answer. Today these are usually made by the big model itself, which is slow and expensive. Jev (TypeSafe, via OpenRouter) is a model built only to decide: it returns a choice and a probability, never text. The project asks when that is worth it.

## The deciders (the same set at every decision point)

| Decider                | What it is                                                   | Role                    |
|------------------------|--------------------------------------------------------------|-------------------------|
| Jev                    | Decision-only model, API (~\$0.00002 per decision, ~0.15 s)  | under test              |
| Kev-4B                 | Open-weights Jev-like model, run on our A100                 | open stand-in (not Jev) |
| GPT-5.6                | Frontier LLM asked the same question as a forced choice      | ceiling                 |
| GPT-OSS-20B / Qwen3-8B | Small LLMs (API / our A100)                                  | cheap-LLM baseline      |
| Floor                  | Small classifier trained on the task's own labelled data     | "train your own"        |
| Task baselines         | Similarity threshold, vCache, always-small/large, random mix | what people use today   |

## The three decision points

| Decision point | The decision                                      | Data                                     |
|----------------|---------------------------------------------------|------------------------------------------|
| Agent control  | Which tool should an agent call next (or reply)?  | tau2-bench airline, 50 tasks             |
| Cache guard    | Can a cached answer be served for this new query? | GSM-Plus, vCache LMArena + SearchQueries |
| Router         | Small model (Mixtral) or large model (GPT-4)?     | RouterBench, 1,998 prompts               |

## How we measure

- Decision-only replay: every decider answers the same saved situations, 3–5 times each, graded against a known right answer. Accuracy, \$ per decision, latency, and GPU energy for local models.
- In the loop (agent control only): full episodes where the decision is acted on, graded by the benchmark.
- Fair comparisons: every number has a 95% bootstrap CI; claims are paired differences on the same items; anything tuned (thresholds, cascades, classifiers) is tuned on a separate dev split, never on test.

## Decision point 3 · Agent control

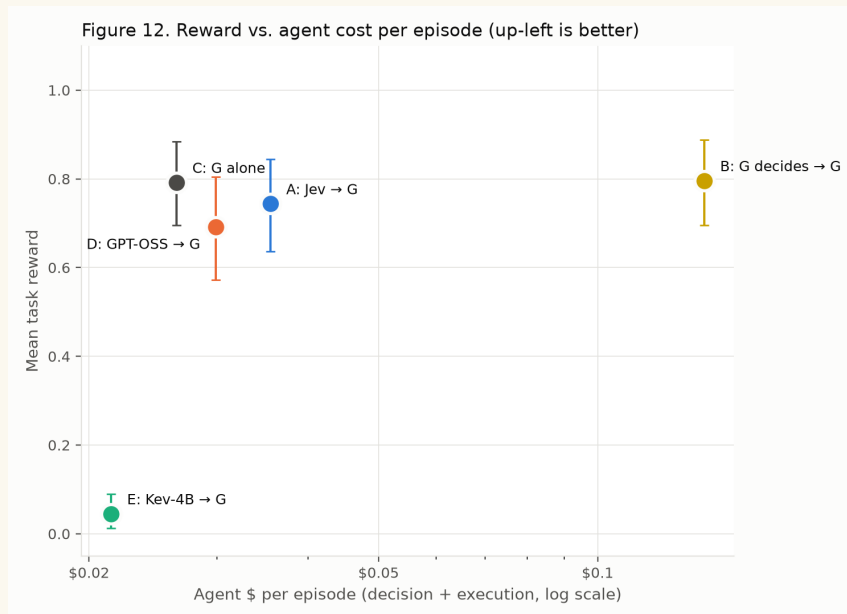
An airline-support agent (tau2-bench) must pick its next action at each turn. Arm 0 replays 1,076 saved states; arms A-E run full episodes with GPT-5.6 as the executor G and the simulated customer.

|                | <i>Jev</i> | <i>GPT-5.6</i> | <i>Jev -&gt; GPT-5.6 cascade</i> | <i>GPT-OSS-20B</i> | <i>Kev-4B</i> |
|----------------|------------|----------------|----------------------------------|--------------------|---------------|
| Arm 0 accuracy | 81.4%      | 88.5%†         | 84.9%                            | 67.0%              | 32.1%         |
| \$ / decision  | 1.7e-4     | 3.8e-3         | 8.2e-4                           | 3.1e-4             | local         |

† graded against its own reference actions: an upper bound.

| <i>In the loop (250 episodes each)</i> | <i>A: Jev -&gt; G</i> | <i>B: G decides -&gt; G</i> | <i>C: G alone</i> | <i>D: GPT-OSS -&gt; G</i> | <i>E: Kev -&gt; G</i> |
|----------------------------------------|-----------------------|-----------------------------|-------------------|---------------------------|-----------------------|
| Reward                                 | 0.744                 | 0.796                       | 0.792             | 0.692                     | 0.045                 |
| Agent \$ / success                     | \$0.048               | \$0.176                     | \$0.033           | \$0.043                   | \$0.476               |

Why no win here: the tool choice comes right before generation that happens anyway (G still writes the tool's arguments), so a separate decider only adds a step. Jev is far cheaper than G deciding separately (A - B: -\$0.105 and -24 s per episode, -5.2 pts reward), but G's own single call (C) is cheaper still.



## Decision point 2 · Cache guard

A semantic cache found an earlier query similar to the new one: reuse its answer, or regenerate? Here the decision replaces generation: a right reuse skips the model call. 2,000 test queries per dataset.

| <i>Accuracy</i>                  | <i>GSM-Plus<br/>(number changed?)</i> | <i>LMarena<br/>(chat rewordings)</i> | <i>SearchQueries<br/>(real searches)</i> |
|----------------------------------|---------------------------------------|--------------------------------------|------------------------------------------|
| Jev                              | 93.3%                                 | 71.2%                                | 71.3%                                    |
| Threshold -> Jev cascade         | 93.3%                                 | 92.6%                                | 70.2%                                    |
| GPT-5.6                          | 96.9%                                 | 70.6%                                | 71.0%                                    |
| Similarity threshold (dev-tuned) | 66.1%                                 | 93.0%                                | 70.3%                                    |
| vCache (delta = 0.05)            | 66.1%                                 | 58.6%                                | 54.0%                                    |
| Floor (trained on dev)           | 81.7%                                 | 90.9%                                | 67.8%                                    |

Reading it: on GSM-Plus a changed number barely changes the wording, so every similarity method can only refuse to reuse (66% = never reuse); Jev reads the question and catches it (+27 pts, paired). On LMArena rewordings really are similar, so the free threshold wins; the cascade (threshold first, Jev only when unsure) ties it and keeps Jev's GSM-Plus win: the best design overall. SearchQueries: every decider sits at 70-72%, GPT-5.6 included, which points at noisy labels (your audit).

| <i>\$ per query vs no cache</i> | <i>GSM-Plus (no cache \$0.0016)</i> | <i>LMarena (no cache \$0.0084)</i> |
|---------------------------------|-------------------------------------|------------------------------------|
| Jev                             | 72%                                 | 32%                                |
| Threshold -> Jev                | 72%                                 | 6%                                 |
| Threshold                       | 100%                                | 5%                                 |
| GPT-5.6 as the guard            | 136% (worse than no cache)          | 55%                                |

Figure 15. Cache guard: reuse vs. wrong reuse (up and to the left is better)

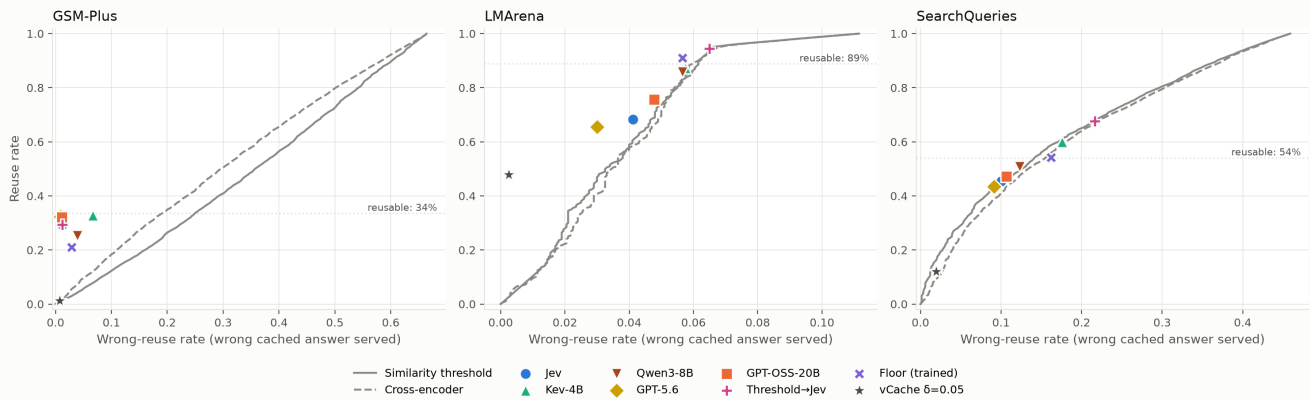
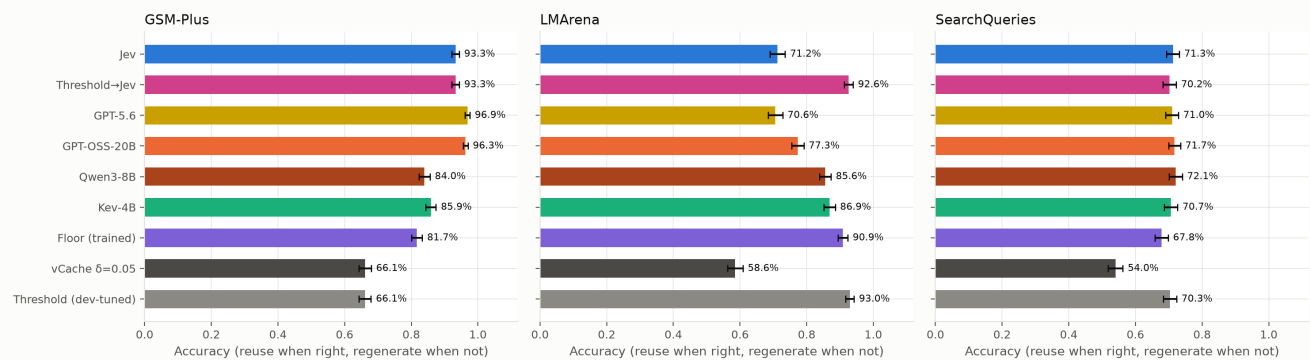


Figure 15. Up and to the left is better. On GSM-Plus every LLM decoder sits far above the similarity curves; on SearchQueries everything sits on the threshold curve.

Figure 16. Cache-guard accuracy with 95% CI (clustered by class)

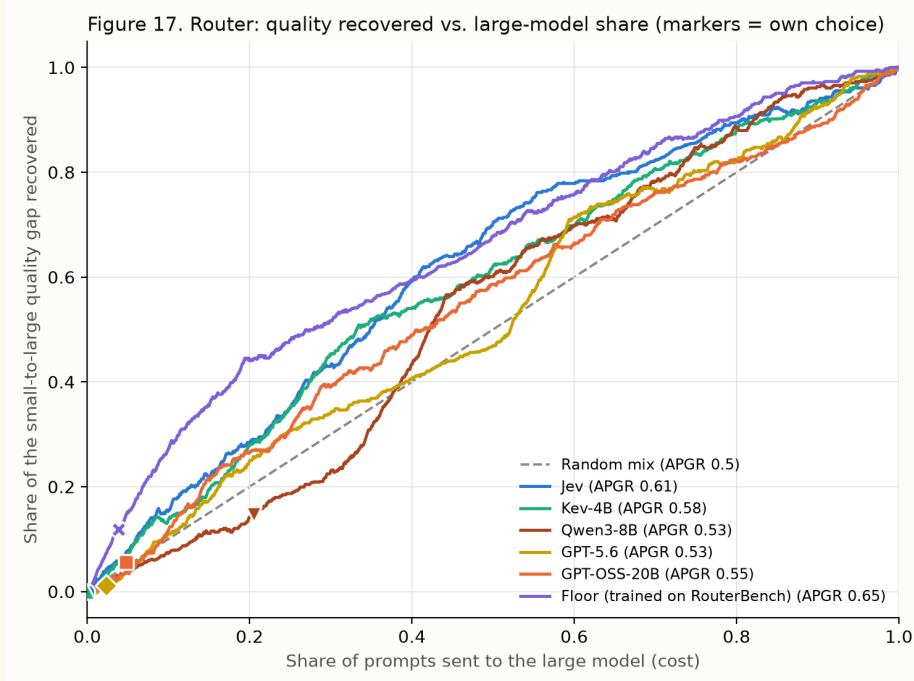


## Decision point 1 · Router

Send each prompt to the small model (Mixtral-8x7B) or the large one (GPT-4, 10–50× the cost)? RouterBench records both models' graded answers, so only the decisions are paid. Always-small scores 0.589, always-large 0.792; 28% of prompts need the large model. APGR = share of that gap a router recovers, averaged over every budget; a random mix scores 0.5.

| Router                                     | APGR [95% CI]        | – Jev [95% CI]          | Own choice: sent large |
|--------------------------------------------|----------------------|-------------------------|------------------------|
| Floor (trained on 20k RouterBench prompts) | 0.649 [0.621, 0.678] | +0.040 [+0.001, +0.070] | 3.9%                   |
| Jev                                        | 0.609 [0.585, 0.642] | –                       | 0%                     |
| Kev-4B                                     | 0.582 [0.553, 0.609] | –0.027 [–0.055, –0.007] | 0%                     |
| GPT-OSS-20B                                | 0.546 [0.516, 0.576] | –0.063 [–0.102, –0.034] | 4.7%                   |
| GPT-5.6                                    | 0.529 [0.510, 0.568] | –0.080 [–0.106, –0.045] | 2.3%                   |
| Qwen3-8B                                   | 0.531 [0.488, 0.540] | –0.078 [–0.130, –0.067] | 20.6%                  |

Why no clear win: routing asks "will this small model get this right?", and that depends on Mixtral's particular weaknesses, which are not visible in the prompt. Only a router trained on Mixtral's own record does better than Jev, and only just (+0.04, CI barely above 0). Jev is the best untrained router (8 pts ahead of GPT-5.6), but its own choice is always "small": using it needs a threshold tuned on dev.



## Putting it together

| Decision point        | Decision replaces generation? | Decider sees what the cheap option can't?     | Pays off?                       |
|-----------------------|-------------------------------|-----------------------------------------------|---------------------------------|
| Agent control         | No: G generates anyway        | —                                             | No (G's single call cheapest)   |
| Cache guard: GSM-Plus | Yes                           | Yes: the changed number                       | Yes (93% vs 66%; -28% cost)     |
| Cache guard: LMArena  | Yes                           | No: similarity already sees it                | Threshold wins; cascade ties it |
| Router                | Yes                           | No: Mixtral's weaknesses aren't in the prompt | Barely (trained router best)    |

The thesis: a cheap standalone decider pays off when its decision replaces generation and it can see something the cheap alternative cannot. Two corollaries: (1) a frontier model is a poor decider for these jobs: as a cache guard it costs more than answering, and as a router it is at chance; (2) the best deployment is a cascade: the free rule decides the clear cases, the decider only the ambiguous ones.

### Caveats the paper must state

- Kev is an open reproduction of the System One idea, not Jev's weights.
- GPT-5.6's agent-control accuracy is graded against its own trajectories (an upper bound).
- LMArena and SearchQueries labels were made by GPT-4.1-nano; scores there are provisional until the label audit.
- RouterBench's models are from 2023; the router predicts their recorded correctness.
- Local deciders count as \$0 per decision; their cost is GPU energy (reported separately).

## Part 11 · Handoff since #6

### Where we are

| Step          | What                                                                                       | Status                          |
|---------------|--------------------------------------------------------------------------------------------|---------------------------------|
| Agent control | arm 0, U4 in-loop A-E, U5 figures 1-14                                                     | DONE (handoffs #4-#6)           |
| CG1-CG6, CG8  | cache-guard streams, question, harness, baselines, full run, regeneration cost, figs 15-16 | DONE                            |
| CG7           | label audit of 100 cases (results/replay/cacheguard-label-audit.csv)                       | TODO — needs your hand verdicts |
| R1-R6         | router stream, question, harness, floor, full run, fig 17                                  | DONE                            |
| Rigor pass    | router APGR CIs + dev-tuned operating points; cache-guard paired CIs                       | DONE                            |
| Paper prose   | from figures 1-17 and DECISIONS.md                                                         | TODO                            |

### Spend since #6 (OpenRouter)

| Run                                              | Cost   |
|--------------------------------------------------|--------|
| Cache guard: pilot + full decisions (3 datasets) | \$24.5 |
| Cache guard: regeneration cost (CG6)             | \$2.0  |
| Router: pilot + full + dev runs                  | ~\$8.4 |
| Total                                            | ~\$35  |

Balance left ≈ \$21. Local runs (Kev-4B, Qwen3-8B, floor, cross-encoder, vCache) cost GPU time only.

## Significance (the claims, paired)

Cache guard: paired differences on the same 2,000 test queries, bootstrap over answer classes; \* = 95% CI excludes 0.

### GSM-Plus

| <i>a - b</i>         | <i>queries</i> | <i>accuracy diff [95% CI]</i> | <i>wrong-reuse diff [95% CI]</i> |
|----------------------|----------------|-------------------------------|----------------------------------|
| Jev - Threshold      | 2000           | +27.2* [+25.4, +29.0]         | +0.9* [+0.4, +1.5]               |
| Jev - vCache         | 2000           | +27.1* [+25.3, +29.0]         | +0.5 [-0.1, +1.1]                |
| Jev - Floor          | 2000           | +11.6* [+10.0, +13.2]         | -1.6* [-2.4, -0.8]               |
| Jev - Kev-4B         | 2000           | +7.4* [+6.1, +8.8]            | -5.4* [-6.5, -4.4]               |
| GPT-5.6 - Jev        | 2000           | +3.6* [+2.7, +4.5]            | -0.3 [-0.6, +0.0]                |
| GPT-OSS-20B - Jev    | 2000           | +3.0* [+2.1, +4.0]            | -0.1 [-0.4, +0.3]                |
| Threshold->Jev - Jev | 2000           | +0.0 [+0.0, +0.0]             | +0.0 [+0.0, +0.0]                |

### LMarena

| <i>a - b</i>               | <i>queries</i> | <i>accuracy diff [95% CI]</i> | <i>wrong-reuse diff [95% CI]</i> |
|----------------------------|----------------|-------------------------------|----------------------------------|
| Threshold - Jev            | 2000           | +21.7* [+19.4, +24.0]         | +2.6* [+1.9, +3.4]               |
| Threshold->Jev - Threshold | 2000           | -0.3 [-0.8, +0.1]             | -0.2* [-0.4, -0.0]               |
| Floor - Threshold          | 2000           | -2.1* [-3.1, -1.0]            | -1.1* [-1.6, -0.5]               |
| Kev-4B - Jev               | 2000           | +15.6* [+13.5, +17.7]         | +1.6* [+1.0, +2.3]               |

### SearchQueries

| <i>a - b</i>    | <i>queries</i> | <i>accuracy diff [95% CI]</i> | <i>wrong-reuse diff [95% CI]</i> |
|-----------------|----------------|-------------------------------|----------------------------------|
| Jev - Threshold | 2000           | +0.9 [-1.3, +3.1]             | -11.9* [-13.4, -10.4]            |
| GPT-5.6 - Jev   | 2000           | -0.3 [-1.4, +0.8]             | -0.9* [-1.5, -0.2]               |
| Jev - Floor     | 2000           | +3.5* [+1.3, +5.6]            | -6.2* [-7.7, -4.7]               |

## Router: a deployable operating point

Each router's threshold on P(large) is set on the dev split so ~30% of prompts go to GPT-4 (28% truly need it), then applied unchanged to test. Gain is over a random mix sending the same share.

| <i>router</i> | <i>dev threshold</i> | <i>test: sent large</i> | <i>test quality [95% CI]</i> | <i>vs random mix at that share [95% CI]</i> | <i>\$ / query</i> |
|---------------|----------------------|-------------------------|------------------------------|---------------------------------------------|-------------------|
| Jev           | 0.030                | 28.4%                   | 0.674 [0.655, 0.693]         | +0.027 [+0.015, +0.039]                     | 9.32e-04          |
| Kev-4B        | 0.276                | 29.4%                   | 0.678 [0.660, 0.697]         | +0.030 [+0.018, +0.042]                     | 1.16e-03          |

| <i>router</i>                  | <i>dev threshold</i> | <i>test: sent large</i> | <i>test quality [95% CI]</i> | <i>vs random mix at that share [95% CI]</i> | <i>\$ / query</i> |
|--------------------------------|----------------------|-------------------------|------------------------------|---------------------------------------------|-------------------|
| Qwen3-8B                       | 0.050                | 22.7%                   | 0.624 [0.605, 0.643]         | -0.011 [-0.019, -0.003]                     | 1.67e-03          |
| GPT-5.6                        | 0.010                | 23.9%                   | 0.649 [0.630, 0.669]         | +0.012 [+0.001, +0.024]                     | 2.13e-03          |
| GPT-OSS-20B                    | 0.110                | 30.0%                   | 0.668 [0.649, 0.687]         | +0.018 [+0.007, +0.030]                     | 1.32e-03          |
| Floor (trained on RouterBench) | 0.347                | 28.3%                   | 0.691 [0.673, 0.710]         | +0.045 [+0.033, +0.058]                     | 7.55e-04          |

### ***Router: APGR with paired differences***

| <i>router</i>                  | <i>APGR [95% CI]</i> | <i>- random (0.5) [95% CI]</i> | <i>- Jev [95% CI]</i>   | <i>- floor [95% CI]</i> |
|--------------------------------|----------------------|--------------------------------|-------------------------|-------------------------|
| Jev                            | 0.609 [0.585, 0.642] | +0.109 [+0.085, +0.142]        | —                       | -0.040 [-0.070, -0.001] |
| Kev-4B                         | 0.582 [0.553, 0.609] | +0.082 [+0.053, +0.109]        | -0.027 [-0.055, -0.007] | -0.067 [-0.103, -0.030] |
| Qwen3-8B                       | 0.531 [0.488, 0.540] | +0.031 [-0.012, +0.040]        | -0.078 [-0.130, -0.067] | -0.118 [-0.170, -0.099] |
| GPT-5.6                        | 0.529 [0.510, 0.568] | +0.029 [+0.010, +0.068]        | -0.080 [-0.106, -0.045] | -0.120 [-0.153, -0.070] |
| GPT-OSS-20B                    | 0.546 [0.516, 0.576] | +0.046 [+0.016, +0.076]        | -0.063 [-0.102, -0.034] | -0.103 [-0.145, -0.058] |
| Floor (trained on RouterBench) | 0.649 [0.621, 0.678] | +0.149 [+0.121, +0.178]        | +0.040 [+0.001, +0.070] | —                       |

## What's next

- Label audit (CG7, you, ~1 h): fill your\_verdict for the 100 cases, ideally before reading the two "says" columns. Then: label error rate and corrected LMArena / SearchQueries scores.
- Optional experiment (~\$15-20): the in-loop Jev → GPT-5.6 cascade for agent control (arm 0 says it recovers half of Jev's gap; in the loop it might close A's 5-pt deficit at near C's cost). Needs a small cascade arm in run\_inloop.py.
- Paper prose from figures 1-17, the significance tables and DECISIONS.md.

## Gotchas we hit this round

- GTE-large-en-v1.5 under transformers 5 needs three shims (build from config, restore rope\_scaling, supply get\_extended\_attention\_mask); our embeddings are checked functionally against vCache's (99.7% same-class NN both).
- vCache's code is CC BY-NC-ND: we drive it unmodified from its own env (/scratch/.../dwg-data/vcache\_env), never copy it.
- RouterBench ships pickles: inspected with pickletools (pandas/numpy only) before loading once; kept as parquet.
- Labels can cap scores: on SearchQueries every decider, GPT-5.6 included, hits 70-72%; the first audit cases show GPT-4.1-nano artifacts and plain label errors.
- A frontier model is a self-defeating guard: asking GPT-5.6 "reuse?" costs more than answering.
- Session restarts can re-run committed work; check git log first.

## New files this round

```
src/dwg/cacheguard.py, src/dwg/router.py # the two decision points' questions
scripts/build_cacheguard_streams.py, build_router_streams.py # data
scripts/replay_decisions.py --task cacheguard|router # the deciders
scripts/cacheguard_{crossencoder,floor,vcache,regen_cost,audit}.py, router_floor.py #
baselines
scripts/analyze_cacheguard.py, compare_cacheguard.py, analyze_router.py # analysis
scripts/make_cacheguard_figures.py # figs 15-16 (fig 17 in analyze_router.py)
results/states/cacheguard-*.jsonl, router-routerbench.jsonl; results/replay/cacheguard-*,
router-*
```